

Modernización de Ecosistemas IoT: Un Enfoque Basado en Arquitecturas Orientadas a Eventos, Microservicios y Computación en el Borde

Anibal Alvarado Andrade*

*Servicio Nacional de Aprendizaje - SENA, Neiva, Colombia — anibalalvaradoandrade@gmail.com

Resumen—Este artículo presenta un estudio de investigación aplicada sobre la utilización de Arquitecturas Orientadas a Eventos (EDA) y microservicios como remplazo a arquitecturas obsoletas en el desarrollo de software moderno. La investigación se centra específicamente en ecosistemas de Internet de las Cosas (IoT), debido a que en este tipo de ecosistemas el rendimiento y el procesamiento de datos en tiempo real a grandes velocidades es vital, lo que provoca que el uso de arquitecturas antiguas, genere un sistema obsoleto debido a su bajo rendimiento en el procesamiento continuo de datos y no solo se quedan atrás en tema de rendimiento y procesamiento de datos, sino que los sensores carecen de contexto y de cualquier forma de comunicación externa, lo que impide la comunicación entre sensores y el procesamiento de los datos se hace directamente en la nube, aumentando así los gastos en soluciones en la nube para la mejora del procesamiento de datos y el rendimiento. En este artículo analizamos varias soluciones a estas amenazas, incluyendo arquitecturas orientadas a eventos basadas en microservicios, arquitecturas orientadas a eventos implementando Computación en el Borde (Edge Computing) agregándole a cada uno de los sensores su propio contexto y la posibilidad de comunicarse con otros sensores para la creación incluso de eventos complejos, además de mejorar el rendimiento en el procesamiento de datos, y aún mejor, solucionar uno de los mayores problemas de los sensores IoT que es la normalización de datos heterogéneos. Asimismo, se evalúa el impacto de utilizar mensajería centralizada mediante un bus de eventos (Apache Kafka) para garantizar el desacoplamiento y la comunicación asíncrona entre componentes. Los resultados que se obtuvieron demuestran una mejora significativa en el rendimiento y eficiencia en los sistemas IoT cuando adoptaron arquitecturas orientadas a eventos y microservicios, en comparación con las arquitecturas tradicionales. También se observó una reducción notable en los costos relacionados con el procesamiento de datos en la nube.

Index Terms—Arquitectura Orientada a Eventos, Internet de las Cosas (IoT), Microservicios, Computación en el Borde, Apache Kafka, Procesamiento en tiempo real

I. INTRODUCCIÓN

En la contemporaneidad tecnológica, la inmediatez ha dejado de ser una ventaja competitiva para convertirse en un requisito funcional innegociable. Desde la gestión de cámaras de tráfico en las intersecciones más congestionadas de una metrópolis hasta la operación crítica de satélites

que deben maniobrar autónomamente para esquivar basura espacial en órbita, el volumen y la velocidad de los datos han superado las capacidades físicas de los sistemas tradicionales. Durante décadas, la industria del software fundamentó sus operaciones en arquitecturas monolíticas y en el patrón de comunicación sincrónico de petición-respuesta (Request-Response). Si bien este modelo fue suficiente para la web estática, hoy se ha convertido en el principal cuello de botella para la innovación en el Internet de las Cosas (IoT). Nos enfrentamos a un escenario donde los sensores y sistemas de vigilancia no pueden darse el "lujo" de esperar segundos a que un servidor centralizado procese su solicitud; la realidad exige una reacción en tiempo real.

Este documento nace de la necesidad imperiosa de explorar cómo la ingeniería de software está respondiendo a este colapso del modelo tradicional. A través de un análisis exhaustivo de veinte investigaciones recientes, evidenciamos un cambio de paradigma radical: la transición desde sistemas acoplados hacia *Arquitecturas Orientadas a Eventos* (EDA) y el uso de *Microservicios Asíncronos*. La premisa fundamental que guía este estudio es que el futuro del software no reside en la acumulación pasiva de información en bases de datos estáticas, sino en la capacidad de capturar, interpretar y reaccionar ante los "eventos" que definen nuestra realidad en el instante exacto en que ocurren.

Sin embargo, la implementación de este nuevo paradigma enfrenta desafíos estructurales profundos. Uno de los problemas más críticos es la ineficiencia de las metodologías actuales de implementación en la nube. Como señalan Shujaat et al. [1], el enfoque convencional de Cloud-Centric IoT (CC-IoT) adolece de tiempos de descubrimiento de servicios lentos y una gestión de recursos subóptima. Su investigación destaca que registrar sensores en listas planas sin una estructura lógica (como Árboles de Acceso a Servicios) provoca latencias inaceptables cuando el número de dispositivos escala a millones. Esto sugiere que la solución no es solo "mover datos más rápido", sino organizarlos de manera inteligente antes de que salgan del dispositivo.

A esta complejidad se suma la necesidad de una respuesta contextual. No basta con detectar un evento; el sistema debe saber qué hacer con él según el entorno. Ward et al. [2] introducen una distinción vital con su concepto de

“Arquitectura de Respuesta Dirigida por Eventos”(EDRA). Argumentan que los sistemas actuales son buenos para alertar, pero malos para actuar. Un sistema verdaderamente inteligente debe mantener un “estado de contexto”del usuario o del entorno para decidir si una alerta requiere una acción inmediata o si puede ser ignorada, cerrando la brecha entre la detección técnica y la utilidad del negocio.

Otro obstáculo recurrente que abordaremos es la desconexión semántica y operativa en entornos urbanos. Como bien señalan Phuttharak y Loke [3], y reafirmado por Visnjic et al. [4], las ciudades inteligentes actuales son, en realidad, archipiélagos de datos desconectados. Los sensores actúan de forma aislada, incapaces de comunicarse entre sí debido a la heterogeneidad de sus formatos y protocolos. Sin una arquitectura de plataforma que estandarice y orqueste estos eventos, la promesa de la *Smart City* se diluye en silos de información inútil. De forma similar, en el ámbito de la salud, veremos cómo la coordinación de ambulancias puede dejar de depender de reportes manuales propensos al error humano, pasando a sistemas de fusión de datos basados en eventos complejos que optimizan tiempos de respuesta vitales [5].

La infraestructura subyacente también juega un papel crucial. Park et al. [6] demuestran que es posible construir arquitecturas resilientes y de bajo costo (REDA) integrando protocolos ligeros como MQTT con buses de eventos robustos como Kafka, desafiando la noción de que la alta disponibilidad requiere inversiones millonarias en hardware propietario.

No obstante, es preciso aclarar que esta evolución no está exenta de fricciones. La migración de sistemas monolíticos legados a ecosistemas distribuidos es una tarea costosa y técnicamente demandante. La experiencia de Tecgraf/PUC-Rio al modernizar su sistema de monitoreo masivo, documentada por Laigner et al. [7], sirve como una advertencia sobre la complejidad de gestionar la consistencia de datos cuando se abandona la seguridad de las transacciones ACID tradicionales. Además, adoptar ciegamente estas nuevas arquitecturas sin una estrategia clara de observabilidad puede generar “puntos ciegos” peligrosos donde los errores se vuelven imposibles de rastrear, como advierten Lazzari y Farias [8].

A lo largo de este artículo, desglosaremos cómo tecnologías emergentes se integran en este ecosistema. Analizaremos desde el uso de cámaras neuromórficas y algoritmos de visión como Stack-CNN para la prevención de colisiones espaciales [9], hasta la detección de infracciones de tráfico mediante YOLOv5 en el borde [10]. El objetivo final es proporcionar una visión holística de cómo la arquitectura de software está evolucionando para sobrevivir en un mundo donde el tiempo de respuesta se mide en milisegundos.

II. MARCO TEÓRICO Y TRABAJOS RELACIONADOS

Una cosa es reconocer la urgencia de procesar datos en tiempo real y otra muy distinta es diseñar la infraestructura capaz de soportarlo. Si en la sección anterior

establecimos el “porqué”del cambio hacia arquitecturas orientadas a eventos (EDA), aquí es necesario diseccionar el cómo”. La literatura técnica reciente sugiere que la solución no es una “bala de plata”, sino una convergencia de múltiples frentes: la descentralización del cómputo, la ruptura de los monolitos heredados y la estandarización semántica de la información.

II-A. El Colapso del Monolito y la Evolución a Microservicios

El punto de partida de esta discusión es la obsolescencia funcional de los sistemas tradicionales. Laigner et al. [7] documentan un caso paradigmático en Tecgraf/PUC-Rio: su sistema de monitoreo de flotas, construido como un monolito sobre bases de datos relacionales, sufría bloqueos constantes debido a la concurrencia. La lección que extraen es crítica: intentar escalar un monolito mediante hardware es inútil si la arquitectura de software está acoplada sincrónicamente.

En respuesta a esto, Kommera [11] argumenta que la arquitectura EDA ofrece la única salida viable para la escalabilidad masiva. Al desacoplar a los productores de los consumidores, se elimina la espera activa (blocking I/O). Sin embargo, Shi et al. [12] advierten que no basta con fragmentar el sistema; proponen una “Arquitectura Orientada a Servicios Dirigida por Eventos”(EDSOA) para el IoT, demostrando que el modelo clásico de petición-respuesta (HTTP REST) es demasiado rígido para dispositivos que necesitan “disparar y olvidar”sus datos.

II-B. La descentralización obligatoria: Inteligencia en el Borde

La premisa de enviar todos los datos a la nube para su análisis se ha vuelto insostenible, tanto por costos de ancho de banda como por latencia. La tendencia actual, defendida por autores como Baz et al. [13] y Salamon et al. [14], es transformar los dispositivos IoT y gateways en filtros inteligentes. La idea es simple pero potente: el video o la lectura del sensor no debe salir del dispositivo a menos que constituya un evento relevante.

Park et al. [6] complementan esta visión con una implementación práctica de bajo costo (REDA), demostrando que es posible construir infraestructuras resilientes en la nube utilizando protocolos ligeros como MQTT puenteados hacia Kafka, optimizando así el consumo de recursos en redes de sensores inalámbricos.

Esta filosofía exige repensar incluso cómo capturamos la realidad. Un ejemplo fascinante es el trabajo de Coretti, Varile y Bertaina [9], quienes se alejan de las cámaras tradicionales que graban cuadros completos. Su enfoque con visión neuromórfica procesa únicamente los cambios de luminosidad en los píxeles (eventos físicos), reduciendo drásticamente la carga computacional necesaria para tareas críticas como evitar colisiones espaciales.

II-C. Estructuras de Datos para el Caos del IoT

Ahora bien, mover datos rápido no sirve de nada si los sistemas no se entienden entre sí o si la búsqueda de información es lenta. Shujaat et al. [1] identifican que el registro plano de sensores es ineficiente y proponen una estructura de "Árbol de Acceso a Servicios"(SAT) basada en la ubicación geográfica. Esto valida que la optimización no siempre requiere más hardware, sino una mejor organización lógica de los datos.

En el contexto de las ciudades inteligentes, el problema se agrava por la "Torre de Babel" de formatos heterogéneos. Xu et al. [15] critican duramente el enfoque de "silos" donde cada aplicación IoT funciona aislada. Su propuesta de un mecanismo de provisión de servicios busca romper estas barreras, permitiendo que recursos dispares colaboren. En esta misma línea, Visnjic et al. [4] y Phuttharak y Loke [3] sugieren capas de validación y transformación semántica que estandarizan los datos crudos antes de que toquen el motor de eventos, asegurando que la velocidad no comprometa la calidad de la información.

II-D. Orquestación y Respuesta Contextual

Finalmente, llegamos a la columna vertebral del sistema. Si bien herramientas como Apache Kafka son el estándar para mover estos volúmenes de datos, la forma de estructurar los servicios ha evolucionado. Kul y Tashiev [16] ilustran la potencia de descomponer tareas monolíticas (analizar un video) en microservicios paralelos y pequeños (uno mira el color, otro la velocidad), que se comunican asincrónicamente.

Sin embargo, la reactividad no debe ser ciega. Ward et al. [2] introducen el concepto de "Arquitectura de Respuesta Dirigida por Eventos"(EDRA), argumentando que el sistema no solo debe reaccionar al evento, sino al contexto del usuario en ese momento. Esto añade una capa de inteligencia necesaria para evitar que las notificaciones automáticas se conviertan en spam.

Aun así, en sectores críticos como el financiero, la libertad total es peligrosa. Adekunle [17] sugiere un enfoque híbrido prudente: usar motores de procesos (como Camunda) para orquestar los flujos críticos sobre la capa de eventos. Esta cautela resuena con la advertencia de Lazzari y Farias [8], quienes nos recuerdan que la mayor debilidad de estas arquitecturas distribuidas sigue siendo la dificultad para rastrear errores y monitorear la salud del sistema cuando las cosas salen mal.

III. METODOLOGÍA DE INVESTIGACIÓN APLICADA

Para determinar la efectividad real de la arquitectura orientada a eventos (EDA) en el ecosistema IoT moderno, realizamos un "análisis exhaustivo y crítico" del estado del arte. A diferencia de las revisiones bibliográficas convencionales que se limitan a enumerar aportes teóricos, adoptamos un enfoque de "meta-análisis técnico". Nuestro estudio se centró en la "auditoría arquitectónica" de 20 casos de uso implementados entre 2015 y 2025, con el

propósito de identificar patrones transversales de éxito y fracaso al desplegar estas tecnologías en escenarios de alta demanda.

El objetivo central fue deconstruir las soluciones propuestas por los autores para responder una pregunta fundamental: ¿La adopción de EDA resuelve los problemas de física (latencia/ancho de banda) o simplemente traslada la complejidad del código a la infraestructura?

III-A. Protocolo de Selección y Filtrado

Nuestro filtro principal fue la "viabilidad operativa". En un campo saturado de propuestas teóricas, descartamos aquellos estudios que no presentaran pruebas de concepto funcionales o métricas de rendimiento verificables. Seleccionamos la literatura basándonos en tres condiciones de inclusión estrictas:

1. **Crisis del Sistema Legado:** El estudio debía partir de un problema tangible en arquitecturas tradicionales, como los bloqueos por concurrencia en bases de datos relacionales o la inviabilidad de transmitir video crudo a la nube debido a costos de ancho de banda.
2. **Implementación Explícita de EDA:** La solución no podía ser genérica; debía basarse explícitamente en patrones de desacoplamiento (Pub/Sub), microservicios asíncronos o computación en el borde (Edge Computing).
3. **Evidencia de Impacto:** Requerimos datos comparativos "antes y después", ya fuera a través de la reducción de latencia en milisegundos o mediante la validación cualitativa de una migración exitosa, tal como lo documentan Laigner et al. [7] en su reporte sobre sistemas de monitoreo masivo.

III-B. Taxonomía y Categorización de Estudios

Para evitar comparar "peras con manzanas", clasificamos los 20 estudios seleccionados en tres clústeres temáticos que comparten desafíos técnicos similares:

- **Grupo A - Visión Computacional de Alta Velocidad:** Estudios centrados en el procesamiento de imágenes en tiempo real, donde la latencia es crítica. Aquí contrastamos las técnicas de detección de infracciones de tráfico de Perera et al. [10] y Kul y Tashiev [16] frente a los sistemas de evasión de colisiones espaciales de Coretti, Varile y Bertaina [9].
- **Grupo B - Orquestación IoT y Smart Cities:** Investigaciones enfocadas en la interoperabilidad de sensores heterogéneos. Analizamos las propuestas de Phuttharak y Loke [3] sobre capas semánticas y la arquitectura de gateways inteligentes de Salamon et al. [14].
- **Grupo C - Sistemas Críticos y Transaccionales:** Casos donde la pérdida de un evento es inaceptable, como la coordinación de ambulancias [5] o los sistemas financieros [17], evaluando la necesidad de consistencia sobre velocidad pura.

III-C. Estrategia de Análisis Comparativo

Nuestra estrategia no fue lineal. Aplicamos una matriz de análisis cruzado para validar si los patrones arquitectónicos se sostenían en diferentes dominios. Por ejemplo, buscamos correlaciones entre la estrategia de "filtrado en el borde" utilizada en la tierra para reducir el tráfico de red en ciudades inteligentes y su contraparte en el espacio para conservar energía en satélites. Esta comparación nos permitió destilar principios de diseño universales que no dependen del hardware específico.

De igual manera, evaluamos críticamente las estrategias de integración de datos. Comparamos los enfoques de "fuerza bruta" (procesamiento masivo de video en clusters Kafka) contra las propuestas de "enriquecimiento semántico" (pre-procesamiento y estandarización), analizando cuál estrategia ofrecía un mejor retorno de inversión en términos de complejidad operativa vs. utilidad del dato final.

III-D. Dimensiones de Evaluación

Finalmente, para sintetizar la discusión, sometimos cada estudio a una evaluación bajo cuatro dimensiones técnicas:

- **Rendimiento Neto:** ¿La adopción de EDA redujo efectivamente la latencia end-to-end o solo movió el cuello de botella a la cola de mensajes?
- **Interoperabilidad:** ¿Cómo resolvieron los autores la "Torre de Babel" de los datos? Evaluamos si utilizaron estándares abiertos o soluciones propietarias.
- **Costo de Infraestructura:** Analizamos la carga que implica mantener la solución propuesta (ej. necesidad de clusters Kubernetes, Zookeeper) frente al beneficio obtenido.
- **Madurez y Resiliencia:** Examinamos la capacidad del sistema para recuperarse de fallos, considerando la dependencia de herramientas como Kafka y la necesidad de orquestación externa señalada por Adekunle [17].

Este método riguroso nos permitió separar el *hype* comercial de la utilidad técnica real, proporcionando una base sólida para las recomendaciones que presentamos en las secciones siguientes.

IV. IMPLEMENTACIÓN DEL SOFTWARE

El análisis transversal de los 20 casos de estudio revela que la industria ha superado la etapa de experimentación con EDA para consolidar un **stack tecnológico de referencia**. Identificamos que el éxito de estas implementaciones no radica en una sola herramienta, sino en la orquestación precisa entre tres capas lógicas: la ingesta en el borde (Edge), el transporte asíncrono y el procesamiento distribuido.

A continuación, realizamos una deconstrucción profunda de las decisiones de ingeniería críticas que permitieron a sistemas como el de tráfico de Sri Lanka [10] o el de monitoreo de Tecgraf [7] escalar donde sus predecesores fallaron.

IV-A. El Stack de Visión y Detección (Edge)

En la capa de captura, observamos una homogeneización en torno a soluciones de código abierto optimizadas para hardware limitado. La implementación descrita por Perera et al. [10] es paradigmática en cuanto al manejo de concurrencia: utilizan un servidor Edge ejecutando scripts de **Python** con arquitectura *Multi-threading*. Mientras un hilo dedicado captura el flujo de video, otro ejecuta la inferencia utilizando **YOLOv5** para la detección de objetos y **DeepSORT** para el rastreo (tracking) de trayectorias, asegurando que el bloqueo en la inferencia no congele la captura de frames.

Esta combinación es crucial. Como señalan Kul y Tashiev [16], delegar la extracción de características (tipo, color, velocidad) al borde utilizando redes como YOLOv4-tiny permite que lo que viaje por la red no sea video pesado, sino metadatos ligeros (JSON). En casos extremos de eficiencia, Coretti, Varile y Bertaina [9] reemplazan el procesamiento de cuadros convencional por algoritmos **Stack-CNN**, que procesan únicamente los eventos de cambio de polaridad en sensores neuromórficos, eliminando la redundancia visual desde la raíz.

IV-A1. Estrategias de Preprocesamiento Local: Un hallazgo técnico relevante en la implementación de Perera es el uso de técnicas de visión clásica antes de invocar a la IA. Para la detección nocturna, implementan un pipeline de preprocesamiento que incluye conversión a escala de grises, umbralización adaptativa y operaciones morfológicas para eliminar el ruido de los faros de los coches antes de intentar leer la matrícula. Esto demuestra que la implementación robusta no confía ciegamente en la red neuronal, sino que "limpia" la entrada para maximizar la probabilidad de éxito.

IV-B. El Sistema Nervioso: Patrones de Mensajería

Una vez generado el evento, la estrategia de transporte define la resiliencia del sistema. La literatura muestra dos enfoques dominantes dependiendo de la criticidad y el volumen de datos:

1. **Bus de Eventos Estricto (Kafka):** En entornos de alta densidad de datos, como el análisis de video distribuido, Kul y Tashiev [16] implementan **Apache Kafka** sobre contenedores **Docker**. Su innovación radica en el patrón de "topics jerárquicos" "Mergers": un microservicio publica el color, otro la velocidad, y un tercero ("Merger") se suscribe a ambos para fusionar la información en un evento complejo (Type-Color-Speed). Esto garantiza un desacoplamiento total y permite que cada equipo de desarrollo evolucione sus microservicios sin romper a los demás.
2. **Triggers Serverless (Firebase):** Para escenarios más orientados a la gestión administrativa, como el sistema de multas de Perera et al. [10], se opta por arquitecturas basadas en disparadores. La inserción de una infracción en **Firebase Realtime Database** dispara automáticamente una **Cloud Function**

(HTTP Trigger) que notifica al servidor central. Esto elimina la necesidad de mantener un servidor de mensajería dedicado para eventos de baja frecuencia, optimizando costos operativos.

IV-B1. Protocolos de Ingesta: Más allá de Kafka, Park et al. [6] detallan la importancia de la capa de traducción de protocolos. En su implementación de REDA, los sensores de bajo consumo no hablan Kafka nativamente (que usa TCP pesado); en su lugar, comunican vía **MQTT**, un protocolo ligero ideal para redes inestables. Un componente puente ("MQTT-Kafka Bridge") se encarga de ingerir estos mensajes y producirlos en el bus central, actuando como un amortiguador de picos de carga.

IV-C. Persistencia, Orquestación y Consistencia

La transición a microservicios exige repensar el almacenamiento. Laigner et al. [7] documentan cómo el modelo relacional tradicional se convierte en un cuello de botella bajo carga masiva. Su solución para el sistema de monitoreo de flotas fue migrar la capa de persistencia a **Cassandra**, una base de datos NoSQL optimizada para escrituras rápidas, esencial para guardar la telemetría de miles de sensores sin bloqueos de tabla.

Sin embargo, la coreografía de eventos pura puede volverse inmanejable y difícil de rastrear. Adekunle [17] advierte que, en sistemas financieros o críticos, dejar que los servicios reaccionen libremente es riesgoso (infierno de la coreografía). Por ello, la implementación robusta suele incluir un motor de procesos (como **Camunda BPM**) que orquesta los flujos de Kafka. Este motor mantiene el estado de la transacción y asegura que si un evento de infracción detectada no recibe un evento de "multa pagada" en un tiempo determinado, se dispare una alerta de escalado o compensación.

IV-D. Fragmento de código: Productor de Eventos IoT

El siguiente código conceptualiza la lógica de un nodo Edge basada en el patrón de filtrado descrito por Salamon et al. [14]. A diferencia de un sistema tonto que transmite todo lo que ve, este script implementa lógica de negocio local para decidir qué merece ser enviado al bus global, optimizando el ancho de banda.

```
Simula la logica de 'Smart Filtering' en
el borde.
Solo emite eventos si la confianza de la
IA (YOLO) es alta.
"""
nivel_confianza = datos_vision.get('
confidence', 0)
tipo_infraccion = datos_vision.get('type')

# Filtrado en origen: Reduccion de ancho
de banda y ruido
# como sugieren Salamon et al.
if nivel_confianza > 0.85:
    evento = {
        "sensor_id": sensor_id,
        "timestamp": time.time(),
        "evento": "INFRACCION_DETECTADA",
        "detalles": {
            "tipo": tipo_infraccion,
            "confianza": nivel_confianza,
            "coordenadas": datos_vision.
get('bbox')
        }
    }

# Publicacion asincrona al topico
particionado
# La clave de particion podria ser el
sensor_id
producer.send('alertas-traffic', key=
sensor_id, value=evento)
print(f"Evento critico enviado: {
tipo_infraccion}")
else:
    # Los falsos positivos se descartan
localmente
# Esto reduce la carga en el
consumidor central
pass
```

```
1 from kafka import KafkaProducer
2 import json
3 import time
4
5 # Configuración del productor (Edge Node)
6 # Se conecta al cluster Kafka gestionado por
  Zookeeper
7 producer = KafkaProducer(
8     bootstrap_servers=['kafka-broker:9092'],
9     value_serializer=lambda v: json.dumps(v).
      encode('utf-8')
10 )
11
12 def procesar_sensor_trafico(sensor_id,
    datos_vision):
13     """
```

V. EVALUACIÓN Y SÍNTESIS DE RESULTADOS

Para cuantificar el impacto real de la migración hacia arquitecturas EDA, no nos basamos en promedios abstractos ni en escenarios teóricos. Diseñamos un esquema de evaluación riguroso que clasifica los resultados reportados en los 20 estudios bajo cinco indicadores de rendimiento clave (KPIs): **Precisión de Detección, Latencia de Procesamiento, Escalabilidad Horizontal, Eficiencia de Ancho de Banda y Resiliencia Operativa**.

A diferencia de las pruebas de laboratorio controladas, estos datos provienen de implementaciones en "trincheras reales", lo que les otorga un peso estadístico significativo sobre la viabilidad de estas tecnologías en producción crítica.

V-A. Impacto de la Inteligencia Contextual

El dato más revelador no proviene de la velocidad pura, sino de la capacidad del sistema para adaptarse al contexto local. Perera et al. [10] demostraron que utilizar algoritmos de reconocimiento genéricos (OCR estándar) es insuficiente para escenarios urbanos complejos. Aunque su modelo base (YOLOv5m) alcanzó una precisión media

(mAP) del 96.1% en condiciones ideales, la lectura de matrículas en el mundo real caía drásticamente sin lógica de negocio adicional.

Como se observa en la tabla I, la introducción de un algoritmo *ad-hoc* que .^{entiende}.^{el} formato de las matrículas de Sri Lanka y corrige errores comunes (confusión entre '0' y 'O') aumentó la tasa de éxito en 30 puntos porcentuales. Esto valida una premisa fundamental: la capa semántica es tan crítica como el modelo de IA; el algoritmo no solo debe "vergaracteres, debe saber qué esperar.

Tabla I: Comparación de precisión en detección de matrículas (Sri Lanka)

Método de Procesamiento	Detecciones	Precisión (%)
Solo Preproc. + OCR	27 / 50	54%
Preproc. + Algoritmo Ad-hoc + OCR	42 / 50	84%

Este hallazgo se alinea perfectamente con el estudio espacial de Coretti, Varile y Bertaina [9]. Su implementación demostró que el cambio a visión neuromórfica (procesar eventos de cambio de luz en lugar de cuadros completos) permitió mantener la precisión operativa para la detección de basura espacial reduciendo el ruido de fondo casi a cero, algo que las cámaras tradicionales de fotogramas completos no lograron bajo condiciones de iluminación variable en órbita.

V-B. Eficacia de los Modelos de Visión en el Borde

Profundizando en la detección, los resultados de Kul y Tashiev [16] desmienten el mito de que "más grande es mejor.^{en} el Deep Learning. Al comparar variantes de redes neuronales para clasificación de vehículos en tiempo real, descubrieron que el modelo ligero **YOLOv4-tiny** alcanzó una Precisión Promedio (AP) del 98.39%, superando paradójicamente a modelos más pesados y lentos en el dataset de prueba. Esto sugiere que, para tareas específicas de EDA, la velocidad de inferencia del modelo "tinycompensa con creces su menor complejidad teórica.

Además, su análisis sobre la detección de color arrojó una lección técnica importante para el preprocesamiento en el borde: el uso del espacio de color HSV (Hue, Saturation, Value) logró una precisión del 95.63%, sistemáticamente superior al estándar RGB (95.29%). Esto confirma que transformaciones matemáticas simples de los datos en el origen pueden mejorar la calidad de la inferencia sin incurrir en costos computacionales adicionales de hardware.

V-C. Desbloqueo del Rendimiento (Throughput)

En términos de velocidad, la evidencia apunta a que el cuello de botella tradicional no era el procesamiento, sino la capacidad de ingesta de datos. El sistema legado de Tecgraf/PUC-Rio [7] sufría bloqueos constantes (*deadlocks*) por concurrencia al intentar escribir en bases de datos compartidas, limitando severamente su capacidad de rastreo de flotas.

Para probar la escalabilidad de la solución EDA como reemplazo, Kul y Tashiev [16] sometieron su arquitectura

basada en Apache Kafka a pruebas de carga incremental agresivas. Los resultados, detallados en la tabla II, son contundentes: el sistema no solo soportó un aumento exponencial en la carga (de 10 a 100,000 mensajes), sino que el tiempo de consumo se mantuvo extremadamente eficiente. Notablemente, procesar e ingerir 100,000 eventos complejos tomó apenas 1.2 segundos (1209 ms). Esto demuestra que la arquitectura asíncrona puede manejar picos de tráfico ("backpressure") que colapsarían instantáneamente a un servidor HTTP REST tradicional.

Tabla II: Rendimiento de Apache Kafka (Producción vs. Consumo)

Carga (Mensajes)	T. Producción (ms)	T. Consumo (ms)
10	40	54
100	145	259
1,000	285	295
10,000	400	456
100,000	2,863	1,209

V-D. Optimización de Recursos y Ancho de Banda

Un aspecto crítico que a menudo se ignora es el costo de transmisión. Baz et al. [13] reportaron resultados significativos al implementar servicios de video orientados a eventos en entornos centrados en el borde (Edge-Centric). Su estudio muestra que al procesar el video localmente y transmitir solo los metadatos del evento (ej. intruso detectado- timestamp), se reduce el consumo de ancho de banda en más de un 90% en comparación con el streaming continuo a la nube.

De manera similar, Park et al. [6] validaron en su arquitectura REDA que el uso de protocolos ligeros como MQTT para la "última milla" de los sensores, conectado a un bus Kafka central, reduce drásticamente el consumo de energía en los dispositivos IoT, extendiendo su vida útil operativa. Esto contrasta con los hallazgos de Roest y Florkowski [18], quienes advierten que sin esta estrategia de filtrado, la "desconexión semántica" provoca que los sistemas se saturen transmitiendo datos visuales que no aportan información real.

V-E. Eficiencia Operativa y Reducción de Carga Humana

Finalmente, los resultados cualitativos en la toma de decisiones son igual de impactantes. Bruns y Dunkel [5] reportaron que, al implementar Procesamiento de Eventos Complejos (CEP) en la coordinación de ambulancias, se eliminó la necesidad de que los conductores reportaran manualmente sus estados (.^{en} ruta", "libre"). El sistema infiere el estado automáticamente cruzando eventos de GPS y telemetría, reduciendo el error humano y acelerando los tiempos de despacho en situaciones críticas.

En el ámbito de la gestión de infraestructuras masivas, la reorganización lógica de los sensores mediante "Árboles de Acceso a Servicios"(SAT) propuesta por Shujaat et al. [1] mejoró la eficiencia de recuperación de datos (lookup time), demostrando que la optimización no siempre

requiere hardware nuevo, sino estructuras de datos más inteligentes. Asimismo, Xu et al. [15] evidenciaron que romper los "silos" de información mediante un mecanismo de provisión de servicios basado en eventos permite una interoperabilidad que las arquitecturas estáticas no pueden ofrecer.

En resumen, los datos confirman nuestra hipótesis inicial: la mejora es estructural. Los sistemas que adoptaron EDA y filtrado en el borde [14] lograron procesar volúmenes de datos superiores con mayor precisión y menor costo, validando que la asincronía es el único camino viable para la escalabilidad en el IoT moderno.

VI. DISCUSIÓN

La transición hacia Arquitecturas Orientadas a Eventos (EDA) en el contexto del IoT no debe interpretarse simplemente como una actualización tecnológica superficial, sino como un cambio fundamental en la filosofía de diseño de sistemas distribuidos. Al contrastar la evidencia empírica de los 20 estudios analizados, emerge una realidad compleja y llena de matices: si bien descentralizar el cómputo resuelve problemas críticos de física (latencia y ancho de banda) en escenarios de vida o muerte como el tráfico o el espacio [9], [10], el precio a pagar es una carga cognitiva y operativa que muchas organizaciones subestiman peligrosamente. La discusión ya no es si la tecnología funciona, sino si las organizaciones pueden soportar la complejidad que conlleva su implementación correcta.

VI-A. La Paradoja de la Complejidad Operativa

A menudo se vende la idea de que los microservicios simplifican el desarrollo al desacoplar componentes lógicos. Sin embargo, la evidencia analizada sugiere lo contrario en las etapas tempranas de adopción. Kul y Tashiev [16] lograron un rendimiento impresionante de procesamiento (1.2 segundos para 100k eventos), pero esto se consiguió a costa de gestionar una infraestructura pesada: múltiples contenedores Docker, clústeres de Zookeeper y particiones de Kafka que requieren mantenimiento constante.

Esto traslada la complejidad del código (lógica de negocio) a la infraestructura (DevOps). Para un equipo pequeño, mantener una topología de *Kafka Streams* tan compleja como la propuesta en su estudio puede ser más costoso en horas-hombre que el problema de rendimiento que intentan resolver. Laigner et al. [7] confirman esto en su reporte de migración en Tecgraf: abandonar el monolito eliminó los bloqueos de base de datos, pero introdujo la necesidad de gestionar la consistencia eventual, un desafío matemático y lógico que no existía en el modelo anterior. Además, Park et al. [6] señalan que la gestión de múltiples brokers en la nube para garantizar alta disponibilidad incrementa los costos operativos (OPEX) significativamente en comparación con servidores tradicionales, planteando dudas sobre la viabilidad económica para proyectos de escala media.

VI-B. El Abismo Semántico: Datos vs. Información

El verdadero cuello de botella ya no es la capacidad de transmitir datos, sino la capacidad de entenderlos. Roest y Florkowski [18] son tajantes al señalar la "desconexión semántica": un sensor puede enviar terabytes de datos visuales, pero si el sistema no sabe distinguir entre un píxel rojo y un semáforo en rojo, el ancho de banda se desperdicia. La mera transmisión de bytes no constituye comunicación efectiva en un sistema distribuido.

Aquí es donde las propuestas de estandarización cobran vital importancia. Xu et al. [15] y Phuttharak y Loke [3] argumentan que conectar sensores sin una ontología común crea "silos digitales". Su investigación demuestra que sin una capa intermedia que traduzca los dialectos de cada fabricante a un lenguaje de eventos común, la promesa de las "Smart Cities" es inalcanzable. Shujaat et al. [1] profundizan en este problema proponiendo estructuras de "Árboles de Acceso a Servicios" (SAT) para organizar lógicamente los sensores, demostrando que el caos de datos no estructurados ralentiza la recuperación de información crítica, anulando las ganancias de velocidad de la red.

VI-C. El Dilema del Control: Coreografía vs. Orquestación

Existe una tensión no resuelta en la literatura entre la autonomía pura de los servicios y la necesidad de control centralizado. Por un lado, Bruns y Dunkel [5] presentan un caso de éxito rotundo a favor de la autonomía: mediante el Procesamiento de Eventos Complejos (CEP), su sistema infiere el estado de las ambulancias sin intervención humana, reduciendo el error y mejorando los tiempos de respuesta. Este enfoque de inteligencia emergente^{es} ideal para entornos dinámicos donde la rigidez es un defecto.

Por otro lado, en dominios con requisitos legales estrictos, la autonomía es un riesgo. Adekunle [17] advierte que en sistemas financieros no podemos permitir que los eventos fluyan de manera anárquica. Su propuesta de introducir un motor de procesos (BPM) sobre la capa de Kafka sugiere que, a medida que los sistemas maduran, tienden a reintroducir cierto grado de centralización para garantizar la auditabilidad y la consistencia transaccional. Shi et al. [12] apoyan esta visión híbrida con su propuesta de EDSOA, argumentando que los servicios deben tener interfaces de eventos pero mantener un control interno estricto para asegurar la calidad del servicio (QoS).

VI-D. Observabilidad: El Requisito Olvidado

Finalmente, una carencia crítica identificada en la mayoría de las implementaciones es la falta de herramientas de depuración nativas. Lazzari y Farias [8] alertan que un sistema distribuido sin una estrategia de trazabilidad distribuida (*distributed tracing*) es, en la práctica, una "caja negra". Cuando un monolito falla, hay un "stack trace"; cuando una arquitectura de eventos falla, hay silencio o inconsistencia de datos.

La lección para los arquitectos de software es clara: invertir en Kafka o Edge Computing [14] es inútil si no se invierte una cantidad equivalente de recursos en observabilidad. Sin ella, la resolución de incidentes en producción se vuelve exponencialmente más difícil que en las arquitecturas tradicionales. Kommera [11] añade que la falta de visibilidad sobre el flujo de eventos no solo afecta la depuración técnica, sino que impide entender el comportamiento del negocio en tiempo real, perdiendo una de las principales ventajas teóricas de la arquitectura EDA.

VII. CONCLUSIONES Y TRABAJO FUTURO

La presente revisión sistemática permite afirmar categóricamente que la transición hacia Arquitecturas Orientadas a Eventos (EDA) y microservicios en el ecosistema IoT ha dejado de ser una exploración académica para convertirse en una ****necesidad técnica impuesta por la física**** de los datos modernos. La evidencia empírica analizada demuestra que las arquitecturas tradicionales, centralizadas y monolíticas, como la descrita en el caso de estudio de Tecgraf [7], han alcanzado un límite funcional insuperable. Ya no poseen la capacidad elástica ni la velocidad de conmutación necesarias para gestionar la concurrencia que requieren escenarios críticos actuales, como la prevención autónoma de colisiones espaciales mediante visión neuromórfica [9] o el control de tráfico masivo en tiempo real [10].

VII-A. Síntesis de Hallazgos Técnicos

Del análisis cruzado y exhaustivo de los 20 casos de estudio, derivamos cinco pilares fundamentales que definen el estado del arte actual y separan las implementaciones exitosas de los fracasos arquitectónicos:

1. **La supremacía del Borde (Edge):** El modelo de "nube céntrica" se ha vuelto económicamente y técnicamente inviable para aplicaciones de latencia crítica. La implementación de filtrado inteligente en el borde, documentada extensamente por Baz et al. [13] y Salamon et al. [14], valida que procesar el dato "donde nace" no solo reduce la latencia a milisegundos, sino que disminuye drásticamente los costos operativos (OPEX) de ancho de banda al evitar la transmisión de terabytes de ruido digital irrelevante hacia los servidores centrales.
2. **Eficiencia sobre Fuerza Bruta:** No siempre la solución es escalar verticalmente el hardware. Estudios como el de Kul y Tashiev [16] demostraron empíricamente que modelos de IA optimizados y ligeros (como Tiny-YOLO) pueden superar en eficiencia operativa a modelos complejos si se integran correctamente en una arquitectura de flujo paralelo (Kafka Streams). La optimización de la topología del software rinde más frutos que la mera adición de potencia de cómputo.
3. **El valor del Contexto Semántico:** La velocidad es inútil sin interpretación. La mejora del 54% al 84%

en la detección de matrículas reportada por Perera et al. [10] prueba que la lógica de negocio específica (algoritmos *ad-hoc*) es tan crucial como el motor de IA subyacente. Sin una capa que entienda el contexto local y las reglas del dominio, los algoritmos genéricos fallan, generando datos basura a alta velocidad.

4. **Estructuración Lógica de Datos:** La velocidad de red no compensa una mala organización. Shujaat et al. [1] demostraron que en entornos IoT masivos, la latencia de recuperación de datos es el verdadero cuello de botella. Su propuesta de organizar los sensores mediante "Árboles de Acceso a Servicios" (SAT) en lugar de listas planas confirma que la estructura de datos es tan vital como el protocolo de transmisión.
5. **De la Detección a la Respuesta:** Finalmente, identificamos que detectar un evento es solo la mitad de la ecuación. Como argumentan Ward et al. [2], un sistema verdaderamente resiliente debe implementar una **Arquitectura de Respuesta Dirigida por Eventos** (EDRA). No basta con saber que algo ocurrió; el sistema debe tener la inteligencia contextual para decidir automáticamente qué acción tomar, cerrando el ciclo entre la percepción digital y la acción física.

VII-B. Implicaciones Estratégicas para la Industria

Basándonos en los patrones de éxito y fracaso identificados en la literatura, ofrecemos las siguientes directrices estratégicas para arquitectos de software y líderes técnicos que planean una migración:

- **Gestión de la Complejidad Híbrida:** La asincronía introduce una complejidad de depuración significativa. Para procesos que requieren consistencia transaccional estricta (ACID), como pagos o multas legales, se recomienda mantener un control orquestado centralizado, tal como sugiere Adekunle [17]. La industria debe moverse hacia modelos híbridos donde la coreografía de eventos maneja la ingesta masiva, pero la orquestación BPM maneja el estado del negocio.
- **Invertir en la Capa Semántica:** Antes de conectar sensores masivamente, es imperativo definir una ontología común. Como advierten Phuttharak y Loke [3] y Xu et al. [15], la falta de estandarización crea "silos de datos" que, aunque rápidos, son incapaces de cooperar entre sí, anulando el propósito de una ***Smart City***. La integración debe ser semántica, no solo sintáctica.
- **Evolución Incremental (EDSOA):** Para sistemas legados que no pueden ser reescritos desde cero (Brownfield), la adopción progresiva de una Arquitectura Orientada a Servicios Dirigida por Eventos (EDSOA), propuesta por Shi et al. [12], ofrece un camino intermedio viable. Esto permite envolver servicios antiguos con interfaces de eventos, inyectando reactividad sin el riesgo de una reingeniería total ("Big Bang").

VII-C. Impacto Socioeconómico

Más allá del código, esta revisión destaca el impacto tangible de estas arquitecturas en la sociedad. La capacidad de inferir el estado de una ambulancia en tiempo real mediante CEP, como demostraron Bruns y Dunkel [5], no es solo una mejora de eficiencia; se traduce directamente en vidas salvadas al reducir los tiempos de despacho. De igual manera, las plataformas de decisión para ciudades inteligentes propuestas por Visnjic et al. [4] sugieren que la adopción de EDA es un habilitador clave para la sostenibilidad urbana, permitiendo una gestión de recursos basada en la demanda real y no en estimaciones estáticas.

VII-D. Desafíos Abiertos y Trabajo Futuro

A pesar de los avances, la modernización no está exenta de riesgos críticos. Identificamos que la ****observabilidad**** sigue siendo el talón de Aquiles de estas arquitecturas; sin herramientas de trazabilidad distribuida, los sistemas EDA se convierten en cajas negras imposibles de auditar en caso de fallo, un riesgo inaceptable expuesto por Lazzari y Farias [8].

El trabajo a futuro en esta línea de investigación debe concentrarse obsesivamente en tres frentes:

1. **Monitoreo Semántico:** Desarrollar herramientas que permitan rastrear no solo si un mensaje llegó, sino si su contenido (payload) mantuvo su integridad lógica a través de múltiples transformaciones asíncronas en microservicios dispares.
2. **Eficiencia Energética en Protocolos:** Seguir la línea de Park et al. [6] para optimizar el consumo de energía en la transmisión de eventos. A medida que el IoT escala, la huella de carbono de la transmisión de datos se vuelve un factor limitante que requiere protocolos más ligeros que HTTP o TCP estándar.
3. **Interoperabilidad Plug-and-Play:** Establecer estándares de interoperabilidad ligera que no requieran ontologías pesadas, facilitando que dispositivos de diferentes fabricantes colaboren en el borde mediante protocolos ágiles como MQTT, eliminando la fricción de integración actual.

En definitiva, la arquitectura distribuida ha dejado de ser una opción para convertirse en el estándar de supervivencia. Mientras no resolvamos la dificultad de observar y depurar estos sistemas, su adopción masiva fuera de los gigantes tecnológicos seguirá siendo una apuesta arriesgada, donde se intercambia estabilidad por velocidad.

APÉNDICE

Este apartado reúne varias anotaciones que se fueron acumulando durante la revisión de los artículos y que, aunque no encajan muy bien en las secciones principales, ayudan a entender cómo se realizó realmente el análisis. No es un apéndice tradicional lleno de tablas de datos; más bien son recordatorios, observaciones sueltas y pequeños extractos que se fueron guardando para no perder el hilo de la investigación.

Apéndice A. Notas durante la búsqueda bibliográfica

Una de las primeras cosas que se anotó mientras se buscaban los **papers** fue lo difícil que es encontrar implementaciones reales en medio de tanta teoría. Se dejó un pequeño archivo de notas internas para filtrar el ruido. No era documentación formal, solo algo como:

“Ojo con los artículos que dicen ‘propuesta de arquitectura’ en el título. Si no tienen sección de experimentos o resultados con números, descartarlos. Necesitamos ver latencia real, no supuestos.”

Esa nota simple nos ahorró leer docenas de documentos que al final no aportaban pruebas tangibles.

Otra anotación que apareció mucho era simplemente esta frase: “Revisar la fecha de publicación antes de leer. Si es de antes de 2015, probablemente la tecnología (Kafka/Docker) ya cambió demasiado.”

Apéndice B. Observaciones sobre la lectura cruzada

En lugar de colocar aquí una lista de lecturas, se dejó registro de las sensaciones que surgieron al comparar autores tan distintos (como los de tráfico vs. los espaciales). Por ejemplo:

- Cuando un artículo no explicaba cómo manejaba los fallos de red, casi siempre era porque asumían una conexión perfecta. Esos los marcamos con un asterisco de “sospechoso”. - La métrica de “precisión” se volvió un indicador confuso. Unos medían precisión por píxel y otros por objeto detectado. Tuvimos que leer la letra pequeña de las tablas varias veces para no comparar peras con manzanas. - La parte de los diagramas de arquitectura terminó siendo más confiable que el texto. A veces el texto prometía “tiempo real”, pero el diagrama mostraba una base de datos relacional en el medio que claramente iba a ser un cuello de botella.

Hubo otra observación que se repitió varias veces en las notas: “El papel aguanta todo, pero las gráficas de latencia no mienten. Si no hay gráfica de estrés, dudar de la escalabilidad.”

Apéndice C. Comentarios sobre la extracción de datos

Extraer los datos de las tablas originales (como las de Kul o Perera) dejó algunas conclusiones que vale la pena guardar.

Una de ellas fue que los datos de rendimiento a veces estaban escondidos en el texto y no en las tablas. A menudo pasábamos por alto un párrafo, pero al releer encontramos el dato clave de los “1209 ms” que cambiaba toda la interpretación del **paper**.

Otro comentario recurrente en las notas era que los autores rara vez hablan de los costos de infraestructura. Nos tocó inferir la complejidad basándonos en los diagramas (“esto necesita al menos 3 nodos de Kafka”), lo cual fue un ejercicio interesante de ingeniería inversa.

También quedó una lista de “cosas que nos hubiera gustado ver” en los estudios analizados:

- “Ver qué pasa con la latencia si se cae la red 5 segundos y vuelve.” - “Saber cuánto cuesta mantener esos contenedores Docker encendidos 24/7.” - “Comparativas con hardware más barato (Raspberry Pi) en lugar de servidores i9.”

Apéndice D. Notas generales del análisis

Durante la redacción del artículo circularon varios mensajes y recordatorios para mantener la coherencia. Algunos de esos apuntes quedaron guardados y ayudan a entender el enfoque:

- “No repetir tanto que ‘Kafka es rápido’, explicar por qué (asincronía/logs).” - “Cuidado con mezclar los resultados de visión (YOLO) con los de infraestructura. Son capas distintas.” - “El caso de las ambulancias es clave para la parte cualitativa, no olvidarlo en la discusión.” - “Recordar que ‘tiempo real’ en el espacio (Coretti) es más crítico que en un parking.”

Estos comentarios pueden parecer simples, pero ayudan a que la sección de Discusión no fuera solo un resumen aburrido, sino una comparativa real.

Apéndice E. Reflexiones finales

Aquí se dejó un conjunto de observaciones sueltas que aparecieron al final, cuando ya se tenía el panorama completo de los 20 artículos:

- La tecnología está lista, pero la integración es el problema real (la torre de Babel de los datos). - Muchos sistemas funcionan bien no por la IA, sino porque filtran los datos *antes* de usar la IA. - La separación por capas (Edge vs Cloud) es lo único que evitó que estos sistemas colapsaran.

Este tipo de anotaciones fueron la base para escribir las conclusiones y nos recuerdan que, a veces, la arquitectura es más importante que el algoritmo.

REFERENCIAS

- [1] M. Shujaat et al., «Improvements to Implementation Methodology for CC-IoT,» *ResearchGate*, 2021. dirección: https://www.researchgate.net/publication/377222341_Improvements_to_Implementation_Methodology_for_CC-IoT
- [2] P. Ward et al., «Event-Driven Response Architecture for Event-Based Computing,» *ResearchGate*, 2005. dirección: https://www.researchgate.net/publication/221501009_Event-driven_response_architecture_for_event-based_computing
- [3] J. Phuttharak y S. W. Loke, «An Event-Driven Architectural Model for Integrating Heterogeneous Data and Developing Smart City Applications,» *Journal of Sensor and Actuator Networks*, 2023. dirección: <https://www.mdpi.com/2224-2708/12/1/12>
- [4] D. Visnjic et al., «Smart City Decision Making System Based on Event-driven Platform,» *LogI - Scientific Journal on Transport and Logistics*, 2021. dirección: <https://sciendo.com/pdf/10.2478/logi-2021-0010>
- [5] R. Bruns y J. Dunkel, «Using Complex Event Processing to support data fusion for ambulance coordination,» *International Conference on Information Fusion*, 2014. dirección: http://confcats_isif.s3.amazonaws.com/web-files/event/proceedings/html/2014Proceedings/papers/fusion2014_submission_55/paper55.pdf
- [6] K. Park et al., «Design and implementation of a cloud-based event-driven architecture for real-time data processing in wireless sensor networks,» *The Journal of Supercomputing*, 2021. dirección: <https://link.springer.com/article/10.1007/s11227-021-03955-6>
- [7] R. Laigner et al., «From a Monolithic Big Data System to a Microservices Event-Driven Architecture,» *ResearchGate*, 2020. dirección: https://www.researchgate.net/publication/341822188_From_a_Monolithic_Big_Data_System_to_a_Microservices_Event-Driven_Architecture
- [8] L. Lazzari y K. Farias, «Uncovering the Hidden Potential of Event-Driven Architecture: A Research Agenda,» *arXiv preprint arXiv:2308.05270*, 2023. dirección: <https://arxiv.org/abs/2308.05270>
- [9] A. Coretti, M. Varile y M. Bertaina, «An efficient neuromorphic approach for collision avoidance combining Stack-CNN with event cameras,» *arXiv preprint arXiv:2506.16436*, 2025. dirección: <https://arxiv.org/abs/2506.16436>
- [10] H. Perera et al., «Traffic Violation Detection System,» *Journal of Community Development and Humanitarian Action*, 2024. dirección: <https://journals.sjp.ac.lk/index.php/contre/article/view/7390>
- [11] A. Kommera, «The Power of Event-Driven Architecture: Enabling Real-Time Systems and Scalable Solutions,» *ResearchGate*, 2020. dirección: https://www.researchgate.net/publication/385668247_THE_POWER_OF_EVENT-DRIVEN_ARCHITECTURE_ENABLING_REAL-TIME_SYSTEMS_AND_SCALABLE_SOLUTIONS
- [12] R. Shi et al., «An Event-driven Service-oriented Architecture for the Internet of Things Service Execution,» *ResearchGate*, 2015. dirección: https://www.researchgate.net/publication/277641221_An_Event-driven_Service-oriented_Architecture_for_Internet_of_Things_Service_Execution
- [13] A. Baz et al., «Event-Driven Video Services for Monitoring in Edge-Centric Internet of Things Environments,» *Proceedings of the 25th Conference of Open Innovations Association FRUCT*, 2019. dirección: <https://fruct.org/publications/volume-25/fruct25/files/Baz.pdf>

- [14] F. Salamon et al., «Event driven and semantic based approach for data processing on IoT gateway devices,» *Journal of Supercomputing*, 2018. dirección: <https://www.academia.edu/download/109262019/s12652-018-0843-y20231219-1-u5ip6.pdf>
- [15] B. Xu et al., «An Event-Driven Service Provisioning Mechanism for IoT (Internet of Things) System Interaction,» *IEEE Access*, 2016. dirección: <https://ieeexplore.ieee.org/abstract/document/7562353/>
- [16] S. Kul e I. Tashiev, «Event-Based Microservices With Apache Kafka Streams: A Real-Time Vehicle Detection System Based on Type, Color, and Speed Attributes,» *IEEE Access*, 2021. dirección: <https://ieeexplore.ieee.org/abstract/document/9446179>
- [17] B. Adekunle, «Designing Event-Driven Architecture for Financial Systems Using Kafka, Camunda BPM, and Process Engines,» *ResearchGate*, 2024. dirección: https://www.researchgate.net/publication/392081813_Designing_Event-Driven_Architecture_for_Financial_Systems_Using_Kafka_Camunda_BPM_and_Process_Engines
- [18] M. Roest y S. Florkowski, «Challenges with Image Event Processing,» *Proceedings of the 11th ACM International Conference on Distributed and Event-based Systems*, 2017. dirección: <https://dlwqtxts1xzle7.cloudfront.net/82405097/DEBS2017-Aslam-libre.pdf>